

halfedge：鲁棒几何谓词模块设计文档

src/predicates.rs

halfedge 库

2026-07-05

目录

1 模块定位	2
2 为什么需要鲁棒谓词	2
3 符号约定	3
3.1 orient2d	3
3.2 orient3d (Shewchuk 约定)	3
3.3 tet_signed_volume (几何约定)	4
3.4 incircle / insphere	4
4 核心谓词实现	4
5 点在三角形内判定	4
6 3D 三角形退化判定	5
6.1 算法	5
6.2 投影轴选择公式	5
6.3 流程图	6
6.4 正确性论证	6
7 与旧浮点实现的对比	6
8 调用方迁移清单	7
9 退化守卫汇总	7
10 使用示例	8
11 测试覆盖	9
12 参考资料	9

1 模块定位

`predicates.rs` 是 `halfedge` 库的鲁棒几何谓词模块，基于 `robust crate`（Jonathan Richard Shewchuk 1997 论文的 Rust 直译实现）提供精确的几何方向谓词：

- `orient2d`: 2D 三点方向判定（CCW / CW / 共线）
- `orient3d`: 3D 四点方向判定（在平面下方 / 上方 / 共面）
- `incircle`: 2D 四点共圆判定（在圆内 / 圆外 / 共圆）
- `insphere`: 3D 五点共球判定（在球内 / 球外 / 共球）

以及一批 `bool` / 标量包装函数（`is_ccw2d`、`point_in_triangle_2d`、`tet_signed_volume`、`is_triangle_degenerate_3d` 等）。

使用场景：布尔运算的射线-三角形求交、三角剖分的凸性 / 内点判定、拉伸操作的侧面退化检测、网格自检的退化面识别、网格体积计算等。任何需要符号判定的几何运算都应通过本模块完成。

类别	API	语义
核心谓词	<code>orient2d(a, b, c)</code>	2D 三点方向（CCW > 0 / CW < 0 / 共线 = 0）
	<code>orient3d(a, b, c, d)</code>	3D 四点方向（d 在 abc 平面下方 > 0 / 上方 < 0 / 共面 = 0）
	<code>incircle(a, b, c, d)</code>	2D 共圆（d 在 abc 外接圆内 > 0；要求 abc 呈 CCW）
	<code>insphere(a, b, c, d, e)</code>	3D 共球（e 在 abcd 外接球内 > 0；要求 abcd 正四面体）
2D 便利	<code>signed_area2d(a, b, c)</code>	$\frac{1}{2} \text{orient2d}(a, b, c)$ （CCW 为正）
	<code>triangle_area2d(a, b, c)</code>	2D 三角形无符号面积
	<code>is_ccw2d(a, b, c)</code>	<code>orient2d > 0</code>
	<code>is_collinear2d(a, b, c)</code>	<code>orient2d = 0</code>
	<code>is_convex_vertex2d(a, b, c)</code>	<code>orient2d > 0</code> （ear clipping 凸性判定）
2D 点-三角形	<code>point_in_triangle_2d(p, a, b, c)</code>	4 次 <code>orient2d</code> 同号判定
	退化三角形（共线）	返回 <code>false</code>
3D 便利	<code>tet_signed_volume(a, b, c, d)</code>	$-\frac{1}{6} \text{orient3d}(a, b, c, d)$ （CCW 朝外为正）
	<code>is_coplanar(a, b, c, d)</code>	<code>orient3d = 0</code>
	<code>is_triangle_degenerate_3d(a, b, c)</code>	主轴投影 + <code>orient2d</code> 精确共线判定

2 为什么需要鲁棒谓词

普通 `f64` 浮点运算在退化情况（共线、共面、共圆、共球）附近会因舍入误差给出错误符号，导致依赖符号判定的算法在边界情况下失败：

- 布尔运算：点刚好在三角形边上时被误判为内 / 外；

- 三角剖分：共线点序列被误判为 CCW / CW，导致 ear clipping 死循环；
- 内外判定：射线刚好穿过三角形顶点时方向符号错误；
- 退化检测：浮点面积阈值 10^{-12} 对极小但不退化的三角形误报。

Shewchuk 自适应精度算法的核心思想是：

1. 先用普通 **f64** 计算快速得到一个近似符号；
2. 同时计算一个**误差上界** ε ；
3. 若 $|result| > \varepsilon$ ，符号确定，直接返回；
4. 否则启动**扩展精度**（adaptive precision）重新计算，使用 **expanded double**（将一个 **f64** 拆为非重叠序列的累加形式）消除舍入误差；
5. 最终保证返回值的**符号是精确的**（值仍可能不精确，但符号决定几何关系，已经足够）。

性能：非退化情况下自适应算法与朴素 **f64** 实现相当（仅多一次误差估计）；仅在接近退化时才付出扩展精度的代价。

3 符号约定

3.1 orient2d

$$\text{orient2d}(a, b, c) = \det \begin{pmatrix} b_x - a_x & b_y - a_y \\ c_x - a_x & c_y - a_y \end{pmatrix} = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

- > 0 : c 在有向线 $a \rightarrow b$ 的**左侧**（三点呈 CCW）；
- < 0 : c 在右侧（CW）；
- $= 0$: 三点共线。

返回值绝对值 = 三角形 abc 有向面积的 2 倍。

3.2 orient3d (Shewchuk 约定)

$$\text{orient3d}(a, b, c, d) = \det \begin{pmatrix} b_x - a_x & b_y - a_y & b_z - a_z \\ c_x - a_x & c_y - a_y & c_z - a_z \\ d_x - a_x & d_y - a_y & d_z - a_z \end{pmatrix}.$$

Shewchuk 约定：从 abc 呈 CCW 顺序的一侧（”上方”）看，

- > 0 : d 在平面 abc 的**下方**；
- < 0 : d 在**上方**；
- $= 0$: 四点共面。

返回值绝对值 = 四面体 $abcd$ 有向体积的 6 倍。

3.3 tet_signed_volume (几何约定)

几何上“CCW 朝外为正体积”的约定与 Shewchuk 相反：

$$\text{tet_signed_volume}(a, b, c, d) = -\frac{1}{6} \text{orient3d}(a, b, c, d).$$

取负原因：当 abc 按右手定则给出朝外法向时， d 在四面体内部（即 abc 平面“上方”，朝向法向相反一侧），此时 Shewchuk 的 `orient3d` 返回负值，而几何约定希望该四面体体积为正，故取负。

3.4 incircle / insphere

`incircle` 要求 a, b, c 呈 CCW 顺序；`insphere` 要求 a, b, c, d 呈正方向（即 `orient3d(a, b, c, d) > 0`）。若输入违反该前置条件，结果符号反转但仍一致。

4 核心谓词实现

`predicates.rs` 仅做 API 包装，直接委托给 `robust crate`：

```
1 use robust::{Coord, Coord3D};
2
3 #[inline]
4 pub fn orient2d(a: [f64; 2], b: [f64; 2], c: [f64; 2]) -> f64 {
5     robust::orient2d(
6         Coord { x: a[0], y: a[1] },
7         Coord { x: b[0], y: b[1] },
8         Coord { x: c[0], y: c[1] },
9     )
10 }
11
12 #[inline]
13 pub fn orient3d(a: [f64; 3], b: [f64; 3], c: [f64; 3], d: [f64; 3]) -> f64 {
14     robust::orient3d(
15         Coord3D { x: a[0], y: a[1], z: a[2] },
16         Coord3D { x: b[0], y: b[1], z: b[2] },
17         Coord3D { x: c[0], y: c[1], z: c[2] },
18         Coord3D { x: d[0], y: d[1], z: d[2] },
19     )
20 }
```

`robust crate` 是 Shewchuk 1997 论文 *Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates* 的 Rust 直译实现，由 `georust` 维护，`crates.io` 累计下载 490 万 +，许可 MIT OR Apache-2.0，与 `halfedge` 双许可兼容。

5 点在三角形内判定

`point_in_triangle_2d(p, a, b, c)` 用 4 次 `orient2d` 同号判定：若 abc 为 CCW，则 p 在内部（含边界）当且仅当 p 相对三条有向边 ab 、 bc 、 ca 均在左侧或边上（`orient` ≥ 0 ）。

```

1 pub fn point_in_triangle_2d(
2     p: [f64; 2], a: [f64; 2], b: [f64; 2], c: [f64; 2],
3 ) -> bool {
4     let o1 = orient2d(a, b, p);
5     let o2 = orient2d(b, c, p);
6     let o3 = orient2d(c, a, p);
7     let tri_sign = orient2d(a, b, c);
8     if tri_sign == 0.0 { return false; } //
9     if tri_sign > 0.0 { // CCW
10         o1 >= 0.0 && o2 >= 0.0 && o3 >= 0.0
11     } else { // CW
12         o1 <= 0.0 && o2 <= 0.0 && o3 <= 0.0
13     }
14 }

```

退化处理：三角形 abc 共线 ($\text{tri_sign} = 0$) 时返回 `false`，避免后续 ≥ 0 / ≤ 0 永真导致误判。

6 3D 三角形退化判定

`is_triangle_degenerate_3d(a, b, c)` 精确判定 3D 三点是否共线（含重合）。直接对 3D 三点用 `orient3d` 无法判共线（只能判共面），故采用**主轴投影 + 2D 谓词**的两步法：

6.1 算法

1. 计算法向 $\vec{n} = (\vec{b} - \vec{a}) \times (\vec{c} - \vec{a})$ 的三个分量绝对值 $|n_x|, |n_y|, |n_z|$;
2. 选取**最大分量轴为丢弃轴**，将三角形投影到其余两轴构成的 2D 平面（保证投影面积最大，避免再次退化）;
3. 在 2D 平面上调用 `orient2d` 精确判定共线： $\text{orient2d} = 0 \iff$ 三点共线。

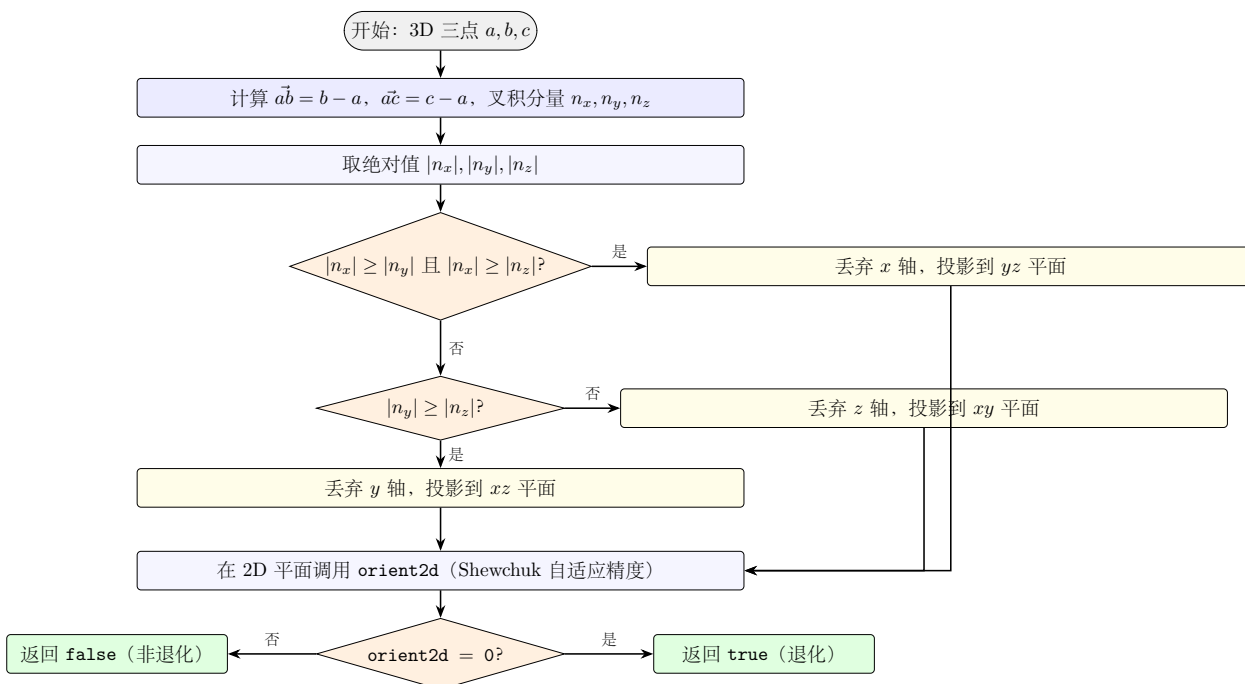
6.2 投影轴选择公式

$$\text{drop_axis} = \begin{cases} x, & |n_x| \geq |n_y| \wedge |n_x| \geq |n_z| \\ y, & |n_y| \geq |n_z| \quad (\text{否则}) \\ z, & \text{否则} \end{cases}$$

对应投影：

- 丢弃 x : $(x, y, z) \mapsto (y, z)$
- 丢弃 y : $(x, y, z) \mapsto (x, z)$
- 丢弃 z : $(x, y, z) \mapsto (x, y)$

6.3 流程图



6.4 正确性论证

为何“最大分量轴”投影能避免再次退化？设原三角形的法向 $\vec{n}$ 在某轴（如 z ）上的分量最大，则将三角形投影到 xy 平面后，新法向 $\vec{n}' = (n_x, n_y)$ 的长度 $\sqrt{n_x^2 + n_y^2}$ 是所有投影中最大的。若 $\vec{n} \neq \vec{0}$ （原三角形非退化），则至少一个分量非零，该最大投影必非零；反之若 $\vec{n} = \vec{0}$ （原三角形退化），所有投影法向均为零，任意轴都判出 $\text{orient2d} = 0$ 。

为何不直接用 $|\vec{n}| < \varepsilon$ ？浮点阈值 ε 难以选取：太小则接近退化的三角形漏报，太大则正常窄三角形误报。Shewchuk 谓词给出精确符号，无需阈值，能在边界情况下做出几何上正确的判定。

7 与旧浮点实现的对比

场景	旧实现（浮点阈值）	新实现（鲁棒谓词）
2D 三点方向	$(b-a) \times (c-a) > 0$	<code>orient2d(a, b, c) > 0</code>
2D 共线判定	$ cross < 10^{-14}$	<code>orient2d == 0</code>
2D 三角形面积	$\frac{1}{2} cross $	<code>signed_area2d</code> （基于 <code>orient2d</code> ）
2D 点在三角形内	朴素叉积同号	<code>point_in_triangle_2d</code> （4 次 <code>orient2d</code> ）
3D 退化三角形	$ cross < 10^{-12}$	<code>is_triangle_degenerate_3d</code>
3D 四面体体积	朴素行列式	<code>tet_signed_volume</code> （基于 <code>orient3d</code> ）
射线-三角形求交	Möller-Trumbore（浮点行列式）	<code>orient3d</code> + 投影 2D + <code>point_in_triangle_2d</code>
拉伸侧面退化	$ edge \times offset < 10^{-12}$	<code>is_triangle_degenerate_3d</code>
网格自检退化面	$area < 10^{-12}$	<code>is_triangle_degenerate_3d</code>

关键改进：旧实现在退化边界（如点恰好共线、面积接近零）附近依赖浮点阈值，存在误报 / 漏报风险；新实现符号判定**精确**，仅在真正退化时报告，避免阈值选取的工程难题。

8 调用方迁移清单

本模块在 `halfedge` 库内的替换位置：

源文件	原函数	替换为
<code>triangulation.rs</code>	<code>v2_cross</code> / <code>signed_area</code>	<code>orient2d</code> / <code>signed_area2d</code>
<code>triangulation.rs</code>	<code>point_in_triangle</code>	<code>point_in_triangle_2d</code>
<code>triangulation.rs</code>	凸顶点 <code>v2_cross ≤ 0</code>	<code>!is_convex_vertex2d</code>
<code>geometry.rs</code>	体积手写行列式	<code>tet_signed_volume</code>
<code>geometry.rs</code>	Möller-Trumbore	<code>orient3d</code> + 投影 2D
<code>boolean.rs</code>	私有 <code>ray_triangle_intersection</code>	调用 <code>geometry::ray_triangle_intersection</code>
<code>validate.rs</code>	<code>area < DEGENERATE_AREA_EPS</code>	<code>is_triangle_degenerate_3d</code>
<code>topology_ops.rs</code>	拉伸侧面 <code>area < 1e-12</code> （2 处）	<code>is_triangle_degenerate_3d</code>

9 退化守卫汇总

API	退化场景	处理
<code>orient2d / orient3d / incircle / insphere</code>	退化输入（共线 / 共面 / 共圆 / 共球）	返回 0.0（精确零，
<code>point_in_triangle_2d</code>	三角形退化（共线）	返回 <code>false</code>
<code>point_in_triangle_2d</code>	p 在边上 / 顶点上	返回 <code>true</code> （含边界
<code>is_triangle_degenerate_3d</code>	三点重合	返回 <code>true</code>
<code>is_triangle_degenerate_3d</code>	法向某分量为零	自动选最大分量轴
<code>tet_signed_volume</code>	四点共面	返回 0.0

10 使用示例

```

1 use halfedge::predicates::*;
2
3 // 1. 2D
4 let a = [0.0, 0.0];
5 let b = [1.0, 0.0];
6 let c = [0.0, 1.0];
7 assert!(orient2d(a, b, c) > 0.0); // CCW
8 assert!(is_ccw2d(a, b, c));
9 assert!((signed_area2d(a, b, c) - 0.5).abs() < 1e-15);
10
11 // 2. f64
12 let p = [1e20, 1.0];
13 let q = [1e20 + 1.0, 1.0];
14 let r = [1e20 + 2.0, 1.0];
15 assert_eq!(orient2d(p, q, r), 0.0); // Shewchuk
16
17 // 3. 2D
18 assert!(point_in_triangle_2d([0.25, 0.25], a, b, c));
19 assert!(!point_in_triangle_2d([0.6, 0.6], a, b, c));
20
21 // 4. 3D
22 let a3 = [0.0, 0.0, 0.0];
23 let b3 = [1.0, 0.0, 0.0];
24 let c3 = [0.0, 1.0, 0.0];
25 let d3 = [0.0, 0.0, 1.0];
26 assert!((tet_signed_volume(a3, b3, c3, d3) - 1.0 / 6.0).abs() < 1e-15);
27
28 // 5. 3D
29 assert!(!is_triangle_degenerate_3d(a3, b3, c3)); //
30 assert!(is_triangle_degenerate_3d(a3, [1.0, 1.0, 1.0], [2.0, 2.0, 2.0])); //
31 assert!(is_triangle_degenerate_3d(a3, a3, a3)); //

```


11 测试覆盖

测试名	验证点
<code>orient2d_ccw</code>	CCW 三点 <code>orient2d > 0</code> , <code>is_ccw2d</code> 为真
<code>orient2d_cw</code>	CW 三点 <code>orient2d < 0</code>
<code>orient2d_collinear_exact</code>	精确共线三点 <code>orient2d = 0</code>
<code>orient2d_collinear_near</code>	浮点易错共线 (10^{20} 量级), 仍精确返回 0
<code>signed_area_2d_unit_triangle</code>	单位三角形面积 = 0.5
<code>point_in_triangle_interior</code>	内部点返回 <code>true</code>
<code>point_in_triangle_vertex</code>	三顶点返回 <code>true</code>
<code>point_in_triangle_edge</code>	边上点返回 <code>true</code>
<code>point_outside_triangle</code>	外部点返回 <code>false</code>
<code>point_in_triangle_cw_orientation</code>	CW 三角形仍能正确判定内点
<code>point_in_triangle_degenerate</code>	共线三角形返回 <code>false</code>
<code>orient3d_basic</code>	Shewchuk 约定: 下方 > 0 / 上方 < 0 / 共面 $= 0$
<code>tet_signed_volume_unit</code>	标准正交四面体体积 = $1/6$
<code>orient3d_coplanar</code>	四点共面 <code>is_coplanar</code> 为真
<code>incircle_inside / outside / on_circle</code>	共圆谓词三类情况
<code>insphere_inside / outside</code>	共球谓词 (要求 <code>abcd</code> 正方向)
<code>convex_vertex_ccw</code>	CCW 凸顶点 / CW 凹顶点
<code>triangle_degenerate_3d_normal</code>	正常 3D 三角形非退化
<code>triangle_degenerate_3d_collinear</code>	3D 共线三点判为退化
<code>triangle_degenerate_3d_coincident</code>	三点重合判为退化
<code>triangle_degenerate_3d_axis_aligned</code>	主轴对齐三角形的退化 / 非退化

`src/predicates.rs` 共 **20** 个单元测试, 全库共 **433** 个。

12 参考资料

- Shewchuk, J. R. (1997). *Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates*. Discrete & Computational Geometry, 18(3), 305–363.
- robust crate 文档: <https://docs.rs/robust>
- Möller-Trumbore 射线求交原始论文 (已被本模块替换): Möller, T., & Trumbore, B. (1997). Fast, Minimum Storage Ray-Triangle Intersection. *Journal of Graphics Tools*, 2(1).